



AI code review & Quality Analyzer

Submitted by:

Aryan

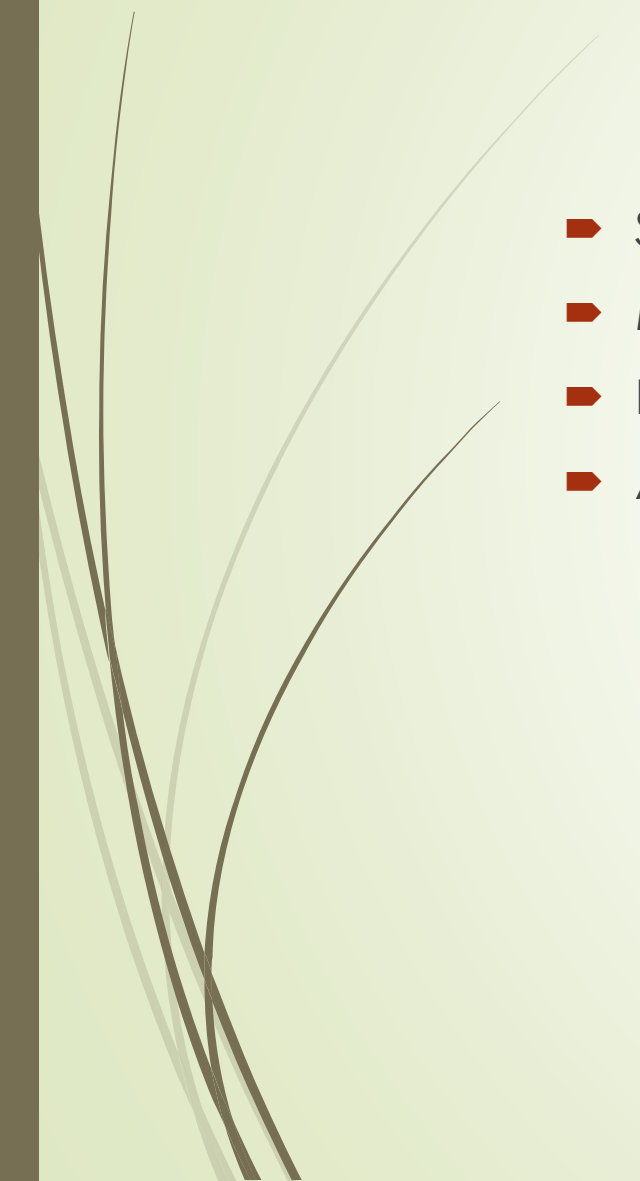
Roll No: 322021013 | 8th Semester

B.Tech (Computer Science & Engineering with

Specialization in Artificial Intelligence & Machine Learning



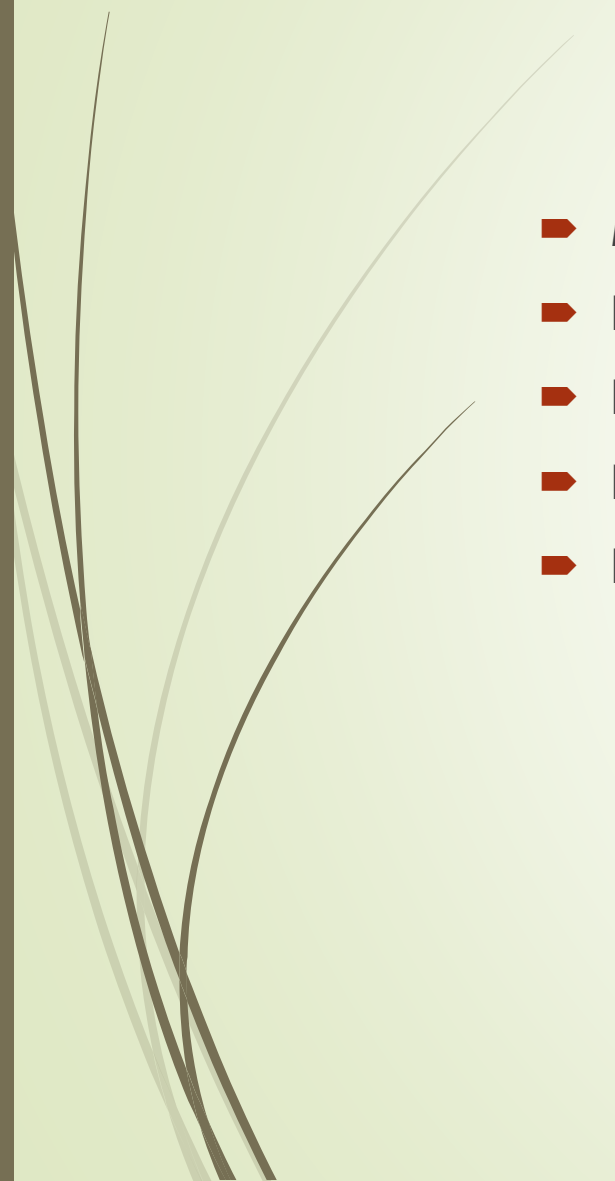
Introduction



- Software quality is crucial for maintainability and reliability.
- Manual code reviews require significant effort.
- Existing tools provide metrics but limited intelligent insights.
- AI and Machine Learning can automate code quality assessment.



Problem Statement

- Manual code review is time consuming.
 - Difficult to analyze large repositories.
 - Lack of intelligent recommendations.
 - Need for automated quality assessment.
 - Existing tools do not provide repository health scores.
- 



Objective

- Analyze Python code automatically.
- Extract software quality metrics.
- Predict code quality using Machine Learning.
- Analyze GitHub repositories.
- Generate repository health score.
- Provide AI generated suggestions.
- Generate PDF reports.



Proposed System

AI-powered platform that:

- Analyzes code.
- Predict Quality.
- Review GitHub repositories.
- Calculates Health scores.
- Generates AI recommendations.
- Create PDF reports.



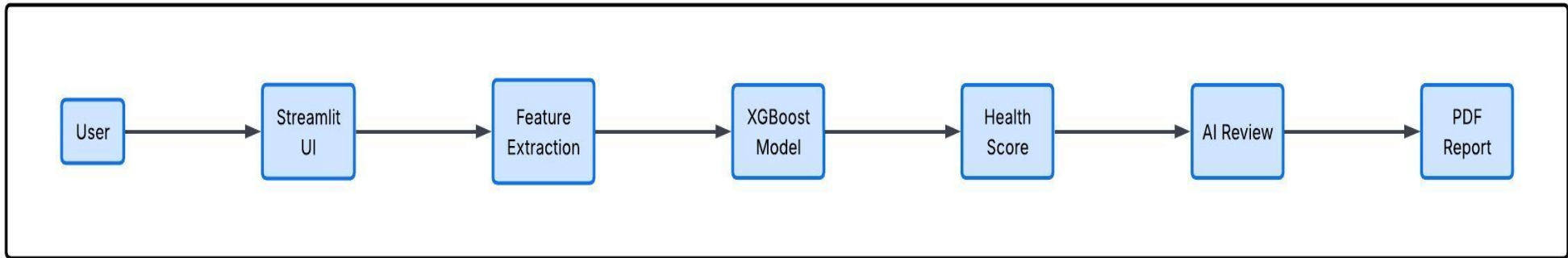
Technology Stack



Technology	Purpose
Python	Development
Streamlit	Frontend
Pandas	Data Processing
XGBoost	ML Model
Plotly	Visualization
Requests	GitHub API
ReportLab	PDF Reports
Groq API	AI Review

System Architecture

Health Scoring App Flow





Working Flow

- Users enters code or GitHub URL
- Feature Extraction
- Quality prediction
- Health Score generation
- AI review
- Visualization
- Pdf report generation



Feature Extraction Module

Extracted Metrics:

- LOC
- SLOC
- LLOC
- Complexity
- Maintainability Index
- Function Count
- Class Count
- Comment Count



Machine Learning Model

XGBoost Classifier

Input:

- ▀ Code metrics

Output:

- ▀ Good
- ▀ Average
- ▀ Poor

Advantages

- ▀ Fast
- ▀ Accurate
- ▀ Handles multiple Features



GitHub Repository Analysis

Features

- URL Parsing
- Python File Detection
- Jupyter Notebook support
- Repository Analysis

Supported Files:

- .py
- .ipynb



File-Level Analysis

Repository files are analyzed individually.

Output:

Files	Quality
app.py	Good
model.py	Average
Utils.py	Poor

Benefits:

- Find weak files
- Improve Functionality



Repository Health Score

Based on:

- Maintainability
- Complexity
- Documentation
- Code Structure

Score range:

Score	Status
80-100	Excellent
60-79	Good
40-59	Average
<40	Poor



AI Review Module

AI Explanation

Generates:

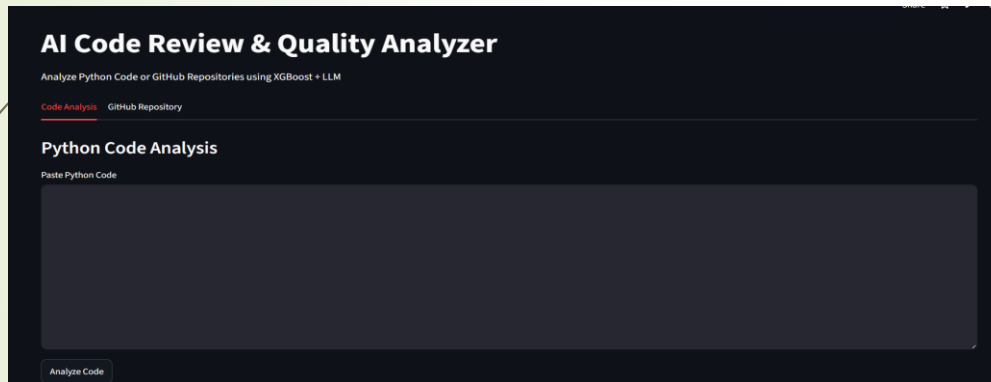
- Strengths
- Weakness
- Recommendations

Example:

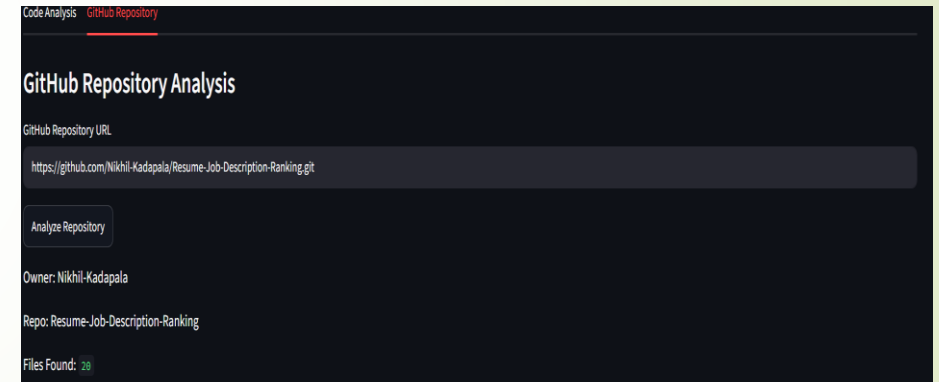
“Code structure is good but complexity should be reduced.”

User Interface Screenshots

Code Analysis

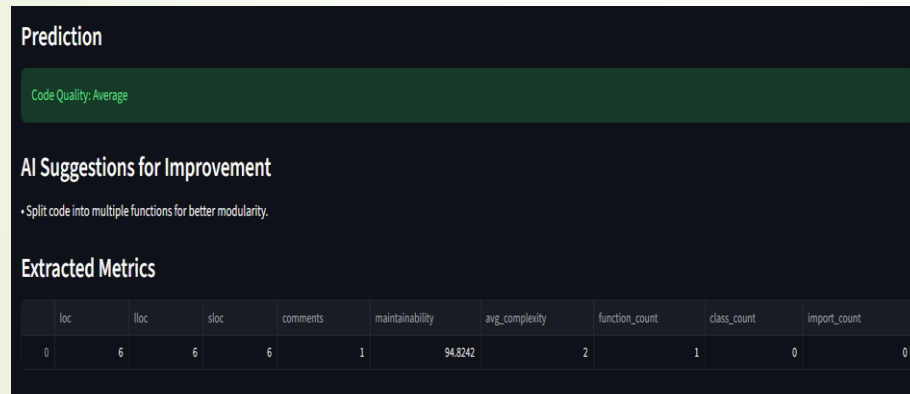


Repository Analysis

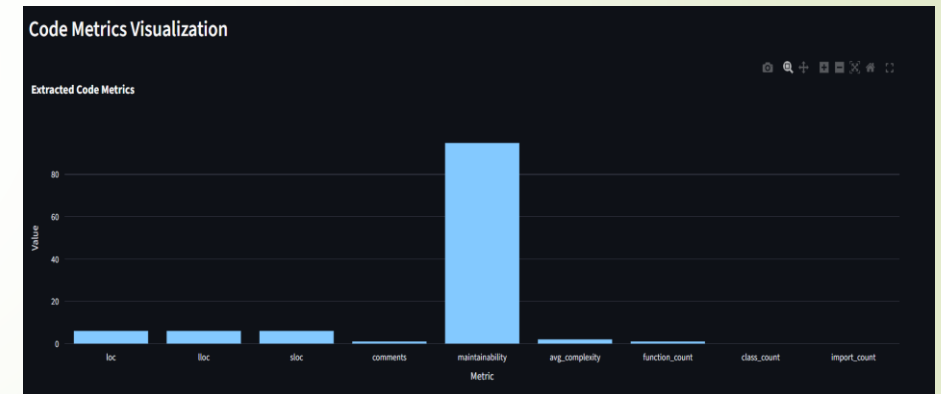


Result Screenshots

Prediction Result



Metrics Chart



Result Screenshots

AI Review

AI Explanation

Code Review

The provided code calculates the factorial of a given number using recursion. Here's a breakdown of the issues and potential improvements:

Issues:

- Input Validation:** The function does not validate the input. It will fail if the input is a negative number or a non-integer. Factorial is only defined for non-negative integers.
- Recursion Limit:** The function uses recursion, which can lead to a stack overflow for large input values. Python has a limit on the depth of recursion to prevent a stack overflow.
- Lack of Documentation:** The function lacks documentation, making it difficult for others to understand its purpose and usage.
- No Error Handling:** The function does not handle any errors that may occur during execution.

Improvements:

- Input Validation:** Add input validation to ensure the input is a non-negative integer.
- Use Iteration:** Replace recursion with iteration to avoid potential stack overflow issues.
- Add Documentation:** Add docstrings to the function to provide documentation and usage information.
- Error Handling:** Add try-except blocks to handle any errors that may occur during execution.

Refactored Code:

```
def factorial(n):
```

File Level Analysis

Found 20 Python files

File-Level Analysis

	File	Quality	Maintainability	Complexity	
0	read_prompts.py	Average		76.55	0
1	check_faithfulness.py	Average		100	2
2	__init__.py	Good		100	0
3	analysis_data.py	Average		100	1
4	jd_data.py	Average		100	1
5	resume_data.py	Average		100	1
6	distill.py	Poor		63.46	5
7	evaluate.py	Poor		41.9	8.5
8	create_finetune_job.py	Poor		45.47	4.5
9	prepare_finetune_data.py	Poor		54.43	3.5



Advantages

- Automated Review
- Faster analysis
- GitHub Integration
- AI recommendation
- PDF reporting
- Easy to use
- Supports notebooks



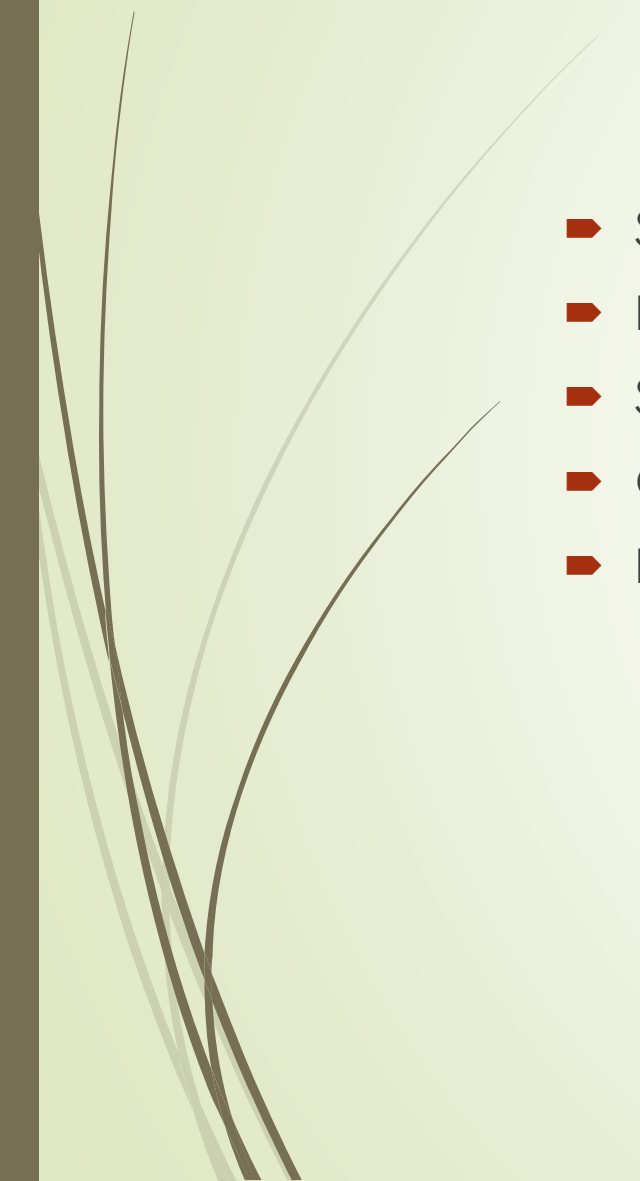
Future Scope

Future Enhancements

- Multi-Language Support
- Security Scanning
- CI/CD integration
- Pull Requests Analysis
- AI Code fixing
- Historical Tracking



Conclusion

- Successfully automated code quality assessment.
 - Integrated Machine Learning and AI.
 - Supports repository-level analysis.
 - Generates intelligent recommendations.
 - Improves software quality evaluation.
- 

Thank You

